

hypersdk Architecture

Hexagonal Multi-Cloud VM Export Platform

A Go daemon with clean hexagonal architecture: 70+ domain types, 17+ port interfaces, 10 cloud provider adapters, job scheduling, carbon-aware migration, cost estimation, RBAC, audit logging, and webhook notifications. Built for enterprise-scale VM export and orchestration.

[Go](#) | [Hexagonal Architecture](#) | [10 Providers](#) | [REST API](#) | [Apache 2.0](#)

Hexagonal Architecture

Clean separation of business logic from infrastructure. Domain has zero external dependencies.



Domain **internal/domain**

70+ canonical Go types — Job, ScheduledJob, VMInfo, ProviderConfig, CarbonIntensity, CostEstimate, Webhook, AuditEvent, Role, Secret. Depends only on Go stdlib (`time` , `encoding/json`). All other packages reference these types. This is the heart of the system.

Ports **internal/ports**

17+ interface contracts — ComputeProvider, ExportProvider, SnapshotProvider, JobStore, ScheduleManager, CarbonProvider, CostCalculator, AuthManager, SecretManager, WebhookSender, AuditLogger, Cache, and more. Defines what adapters must implement.

Adapters **daemon/ + providers/**

Concrete implementations — REST API server, SQLite job store, 10 cloud provider clients, Prometheus metrics, webhook dispatcher, rate limiter, cron scheduler, carbon data fetcher, cost calculator. Swappable without touching business logic.

Domain Layer — Pure Business Types

Zero dependencies. Every type in internal/domain is a plain Go struct — no imports beyond stdlib.

Job & Scheduling

```
type Job struct {
    ID          string
    Status      JobStatus // pending|running|completed|failed|cancelled
    VMName      string
    Provider    ProviderType
    ExportOpts  ExportOptions
    Result      *ExportResult
    CreatedAt   time.Time
    StartedAt   *time.Time
    CompletedAt *time.Time
    Error       string
}

type ScheduledJob struct {
    ID          string
    Schedule     string // Cron format
    JobTemplate  JobDefinition
    AdvancedConfig *AdvancedScheduleConfig
    // Dependencies, retry policy, time windows, priority
}
```

Provider & VM

```
type ProviderType string
const (
    ProviderVSphere = "vsphere"
    ProviderAWS     = "aws"
    ProviderAzure   = "azure"
    ProviderGCP     = "gcp"
    ProviderHyperV  = "hyperv"
    ProviderProxmox = "proxmox"
    ProviderKubeVirt = "kubevirt"
)

type ProviderConfig struct {
    Type      ProviderType
    Endpoint  string
    Username  string
    Password  string
    Region    string
    Timeout   time.Duration
    Metadata  map[string]interface{}}
}
```

Carbon & Cost

```
// Carbon intensity zones
const (
    CarbonZoneUSEast   = "US-CAL-CISO"
    CarbonZoneEUCentral = "DE"
    CarbonZoneAPACTokyo = "JP-TK"
)

// Carbon thresholds (gCO2/kWh)
```

Security & Audit

```
// RBAC roles
const (
    RoleAdmin    = "admin"
    RoleOperator = "operator"
    RoleViewer   = "viewer"
)

// Audit events
```

```
const (  
    CarbonThresholdExcellent = 100.0  
    CarbonThresholdGood      = 200.0  
    CarbonThresholdPoor      = 600.0  
)  
  
// Cloud storage providers for cost calculation  
const (  
    CloudProviderS3          = "s3"  
    CloudProviderAzureBlob   = "azure_blob"  
    CloudProviderGCS         = "gcs"  
)
```

```
const (  
    AuditEventLogin          = "login"  
    AuditEventExportVM       = "export_vm"  
    AuditEventCreateJob      = "create_job"  
    AuditEventCancelJob      = "cancel_job"  
)  
  
type Webhook struct {  
    URL      string  
    Events   []string  
    Headers  map[string]string  
    Retry    int  
    Enabled  bool  
}
```

Ports Layer — Interface Contracts

17+ Go interfaces that define the contracts between domain logic and infrastructure.

Port Interface	Methods	Adapter Implementation	Purpose
ComputeProvider	Connect , ListVMs , GetVM , SearchVMs , Disconnect , ValidateCredentials	providers/vsphere, aws, azure, gcp, hyperv, oci, openstack, alibaba, proxmox, kubevirt	VM discovery
ExportProvider	ExportVM , GetExportCapabilities	Same 10 providers	VM disk export
SnapshotProvider	CreateSnapshot , DeleteSnapshot , ListSnapshots , RevertToSnapshot	Providers with snapshot support	VM snapshots
JobStore	SaveJob , UpdateJob , GetJob , ListJobs , DeleteJob	daemon/store (SQLite)	Job persistence
ScheduleStore	SaveSchedule , UpdateSchedule , GetSchedule , ListSchedules	daemon/store (SQLite)	Schedule persistence
ScheduleManager	Start , Stop , AddScheduledJob , TriggerNow , GetScheduleStats	daemon/scheduler	Cron scheduling
CarbonProvider	GetCurrentIntensity , GetForecast , GetGridStatus , ListZones	providers/carbon (electricitymap, watttime)	Carbon data
CarbonScheduler	GetCarbonStatus , GenerateCarbonReport , EstimateCarbonSavings	daemon/scheduler.CarbonAwareScheduler	Green scheduling
CostCalculator	EstimateCost , CompareProviders , ProjectYearlyCost , EstimateExportSize	providers/cost	Cost estimation
AuthManager	AuthenticateUser , ValidateSession , GetUserRole , Logout	daemon/auth (JWT + API key)	Authentication
SecretManager	Get , Set , Delete , List , Rotate , Health	daemon/secrets (vault, file, env)	Credential storage
WebhookSender	Send , SendConversionStarted , SendConversionCompleted , SendConversionFailed	daemon/webhooks	Event notifications
AuditLogger	Log , Query	daemon/audit.FileLogger	Audit trail

CapabilityDetector	GetCapabilities , GetDefaultMethod	daemon/capabilities	Export method detection
Cache	Get , Set , Delete , Exists , Clear	daemon/cache	Response caching
SystemMetrics	Prometheus counters, gauges, histograms	daemon/metrics	Observability
HTTPMiddleware	Rate limiting, request validation	daemon/ratelimit	API protection

Provider Adapters — 10 Cloud Platforms

Each provider implements ComputeProvider + ExportProvider. Same interface, different clouds.

Provider	Package	Export Method	Auth	Staging	Special Features
VMware vSphere	providers/vsphere	OVF/OVA, VDDK, HTTPS	SSO, user/pass	Direct download	CBT delta sync, snapshot export
AWS EC2	providers/aws	EBS snapshot, AMI export	IAM, access keys	S3 bucket	Cross-region, parallel download
Azure	providers/azure	Managed disk snapshot	Service principal, MSI	Blob storage (SAS)	Parallel download, snapshot-based
GCP	providers/gcp	Persistent disk export	Service account	GCS bucket	Image export, compressed tar.gz
Hyper-V	providers/hyperv	VHD/VHDX direct copy	WinRM, SSH	Direct copy	Checkpoint export
OCI	providers/oci	Boot volume export	API key	Object Storage	Pre-authenticated requests
OpenStack	providers/openstack	Glance image, Cinder	Keystone v3	Swift	Multi-format (qcow2, raw)
Alibaba	providers/alibabacloud	ECS disk snapshot	AccessKey	OSS	Regional export
Proxmox	providers/proxmox	API disk export	Token, PAM	Direct download	Already qcow2/raw
KubeVirt	providers/kubevirt	PVC/DataVolume export	kubeconfig	CDI	Live migration support

Provider Interface (Go Code)

```
// ComputeProvider – VM discovery and management
type ComputeProvider interface {
    Name() string
    Type() domain.ProviderType
    Connect(ctx context.Context, config domain.ProviderConfig) error
    Disconnect(ctx context.Context) error
    ValidateCredentials(ctx context.Context) error
    ListVMs(ctx context.Context, filter domain.VMFilter) ([]*domain.VMInfo, error)
    GetVM(ctx context.Context, identifier string) (*domain.VMInfo, error)
}
```



```
    SearchVMs(ctx context.Context, query string) ([]*domain.VMInfo, error)
}

// ExportProvider – VM disk export
type ExportProvider interface {
    ExportVM(ctx context.Context, id string, opts domain.ExportOptions) (*domain.ExportResult, error)
    GetExportCapabilities() domain.ExportCapabilities
}

// FullProvider – providers that support both discovery and export
type FullProvider interface {
    ComputeProvider
    ExportProvider
}
```

Daemon Services — **daemon/**

The daemon layer contains all infrastructure adapters: REST API, job engine, scheduler, storage, auth, and observability.

daemon/api

- REST API server (net/http)
- OpenAPI 3.0 spec (openapi.yaml)
- JSON request/response
- Versioned routes (/api/v1/...)
- CORS, compression, request ID
- Health + readiness endpoints

daemon/jobs

- Job lifecycle: create, run, cancel
- Concurrent execution with limits
- Progress tracking per job
- Webhook notification on state change
- Implements ports.JobManager
- Automatic cleanup of old jobs

daemon/scheduler

- Cron-based scheduling engine
- Time window enforcement
- Job dependencies (DAG)
- Priority queues (0-100)
- Retry with exponential backoff
- Carbon-aware scheduling mode

daemon/store

- SQLite persistent storage
- Implements JobStore + ScheduleStore
- Schema migrations
- Transaction support
- Query filtering (status, date, provider)
- Backup/restore capability

daemon/auth

- JWT token authentication
- API key authentication
- Session management
- RBAC: admin, operator, viewer
- Password hashing (bcrypt)
- Token rotation and expiry

daemon/rbac

- Role-based access control
- Actions: create, read, update, delete, export, cancel
- Resources: jobs, schedules, providers, config
- Permission matrix per role
- Middleware integration
- Audit log on denied access

daemon/metrics

- Prometheus /metrics endpoint
- Job counters (created, completed, failed)
- Export duration histograms
- Provider latency gauges
- Queue depth metrics
- System resource metrics

daemon/webhooks

- Event-driven notifications
- Slack, Teams, PagerDuty, custom
- Retry with backoff on failure
- Event filtering per webhook
- Conversion started/completed/failed
- Job lifecycle events

daemon/secrets

- Secret management with rotation
- Backends: file, env vars, vault
- Cached secret access
- Automatic rotation support
- Health check for secret backends
- Provider credential storage

daemon/audit

- Full audit trail
- Events: login, export, create/cancel/delete job
- User, IP, timestamp per event
- File-based logger
- Queryable history
- Compliance-ready (SOX, HIPAA)

daemon/cache

- Generic key-value cache
- TTL-based expiration
- VM inventory caching
- Provider response caching
- Reduces cloud API calls
- Configurable cache size

daemon/ratelimit

- HTTP request rate limiting
- Per-user and global limits
- Token bucket algorithm
- Protects cloud API quotas
- 429 Too Many Requests response
- Configurable per endpoint

CLI Binaries — 8 Commands

hypersdk ships 8 purpose-built binaries, all built from cmd/.

Binary	Package	Purpose	Key Features
hypervisord	cmd/hypervisord	Main daemon — runs as systemd service	REST API, job engine, scheduler, auth, metrics, webhooks
hyperctl	cmd/hyperctl	CLI client for the daemon	Job management, provider config, schedule CRUD, status
hyperexport	cmd/hyperexport	Standalone VM export (no daemon)	Direct export from any provider, progress streaming
hyperconvert	cmd/hyperconvert	Disk format conversion	VMDK/VHD/RAW → QCOW2, flatten, compress
hypersdk-dashboard	cmd/hypersdk-dashboard	Web dashboard server	React UI for job monitoring, provider status, metrics
hypersdk-operator	cmd/hypersdk-operator	Kubernetes operator	CRDs for migration jobs, GitOps-friendly, Helm chart
hypersdk-agent	cmd/hypersdk-agent	Remote agent for distributed export	Runs at edge/remote sites, reports to central daemon
hypersdk-control	cmd/hypersdk-control	Control plane management	Multi-node coordination, leader election, HA

How It Fits Together with hyper2kvm

```
User Request (CLI / TUI / API / K8s CRD)
|
+-----v-----+
|  hypervisord   |  hypersdk daemon – multi-cloud VM export
|  (Go daemon)   |  REST API, job queue, scheduler
|                |  Auth, RBAC, audit, metrics
|  10 providers  |  Carbon, cost, webhooks, secrets
+-----+-----+
|  Exported VMDK / VHD / RAW disk image
+-----v-----+
```

```
| h2kvmctl | hyper2kvm – disk conversion + guest fixes
| (Python) | VMCraft engine (qemu-nbd, no libguestfs)
|          | fstab, initramfs, VirtIO, GRUB, SELinux
| zkvm (Go TUI) | Interactive terminal UI
+-----+-----+
| QCOW2 + libvirt XML or KubeVirt YAML
+-----v-----+
| Deploy Target | KVM/libvirt or KubeVirt/Kubernetes
+-----+-----+
```

Carbon, Cost & Scheduling

Enterprise features no competitor offers.

Carbon-Aware Scheduling

- **CarbonProvider port:** Fetches real-time grid carbon intensity from ElectricityMap or WattTime APIs
- **12 grid zones:** US East/West/Central, EU Central/West/North, APAC Singapore/Tokyo/Sydney/India, China Beijing/Shanghai
- **Thresholds:** Excellent (<100 gCO₂/kWh), Good (<200), Moderate (<400), Poor (<600)
- **Scheduling:** Delay migration jobs until carbon intensity drops below threshold
- **Forecasting:** GetForecast returns hourly predictions — schedule for greenest window
- **Reporting:** GenerateCarbonReport shows CO₂ saved by scheduling optimally
- **ESG compliance:** Documented carbon savings for sustainability reporting

Cost Estimation

- **CostCalculator port:** Estimates cloud storage and transfer costs per migration
- **6 cloud providers:** S3, Azure Blob, GCS, OCI, Backblaze, Wasabi
- **Storage tiers:** S3 Standard/IA/Glacier, Azure Hot/Cool/Archive, GCS Standard/Nearline/Coldline
- **EstimateCost:** Per-VM cost including storage, transfer, API calls
- **CompareProviders:** Side-by-side cost comparison across clouds
- **ProjectYearlyCost:** Annual cost projection for ongoing storage
- **EstimateExportSize:** Predicted export size based on disk, format, snapshots

Advanced Scheduling

- **Cron expressions:** Standard 5-field cron for recurring jobs
- **Time windows:** Restrict execution to off-peak hours (e.g., 2AM-6AM)
- **Job dependencies:** DAG-based — job B runs only after job A succeeds
- **Priority queues:** 0 (lowest) to 100 (highest) — critical VMs first
- **Retry policies:** Max retries, exponential backoff, retry delay
- **Concurrency limits:** Max parallel exports per provider
- **TriggerNow:** Manual override to run scheduled job immediately

Observability Stack

- **Prometheus metrics:** /metrics endpoint with job, export, provider metrics
- **Structured logging:** JSON format with trace IDs for request correlation
- **Audit trail:** Every action logged: who, what, when, from where
- **OpenTelemetry:** Distributed tracing via daemon/tracing
- **Dashboard:** React web UI (hypersdk-dashboard) for real-time monitoring
- **Grafana-ready:** Pre-built dashboard templates for common metrics
- **Alerting:** Webhook notifications integrate with PagerDuty/Slack/Teams

Carbon-Aware Scheduling Interface

```
type CarbonProvider interface {
    GetCurrentIntensity(zone string) (*domain.CarbonIntensity, error)
    GetForecast(zone string, hours int) ([]domain.CarbonForecast, error)
    GetGridStatus(zone string, threshold float64) (*domain.GridStatus, error)
    ListZones() ([]string, error)
}

type CarbonScheduler interface {
    GetCarbonStatus(zone string, threshold float64) (*domain.GridStatus, error)
    GenerateCarbonReport(jobID string, start, end time.Time, dataSizeGB float64, zone string) (*domain.CarbonReport, error)
    EstimateCarbonSavings(zone string, dataSizeGB float64, durationHours float64) (map[string]interface{}, error)
}
```

Why This Architecture Matters

Technical advantages of hexagonal design for enterprise migration.

Add a New Provider in Hours, Not Weeks

Implement `ComputeProvider` + `ExportProvider` (6-8 methods). Register in `providers/registry.go`. Done. No changes to daemon, scheduler, API, or any other code. The hexagonal boundary means new providers are isolated, testable, and independently deployable.

Swap Any Adapter Without Risk

Replace SQLite with PostgreSQL? Implement `JobStore` interface. Switch from file-based secrets to HashiCorp Vault? Implement `SecretManager`. Replace Prometheus with Datadog? Implement `SystemMetrics`. Business logic never changes.

Domain Logic Is 100% Testable

`internal/domain` has zero external dependencies — it's pure Go structs with validation logic. Every port interface can be mocked trivially. Integration tests use real adapters. Unit tests use mocks. Full coverage without infrastructure.

Enterprise-Grade Security

RBAC with 3 roles (admin/operator/viewer), 6 actions, resource-level permissions. JWT + API key auth. Secret rotation. Full audit trail (SOX/HIPAA-ready). TLS everywhere. Rate limiting per user and endpoint. All via clean port interfaces.

70+

Domain types
(zero external deps)

17+

Port interfaces
(clean contracts)

10

Cloud provider
adapters

8

CLI binaries
(purpose-built)

Architecture Summary

Domain: 70+ pure Go types — Job, Schedule, VM, Carbon, Cost, RBAC, Audit, Webhook, Secret

Ports: 17+ interfaces — ComputeProvider, ExportProvider, JobStore, CarbonProvider, CostCalculator, AuthManager...

Adapters: 10 cloud providers + daemon services (API, scheduler, store, auth, metrics, webhooks, secrets, audit)

Result: Add providers in hours. Swap adapters without risk. Test everything. Scale to enterprise.